

Núcleo de Situação de Saúde (NSS)

Sprint 1

Zadoque Pires de Deus Souza Carneiro

Agosto de 2026

1 Contexto geral do projeto

O Núcleo de Situação de Saúde (NSS) tem como objetivo centralizar e analisar dados públicos de saúde, com foco inicial em notificações do Sistema de Informação de Agravos de Notificação (SINAN), disponibilizadas via biblioteca PySUS. A partir desses dados, pretende-se monitorar possíveis surtos epidemiológicos, apoiar a elaboração de planos de contingência, servir de base para pesquisas acadêmicas e fornecer informações consolidadas para gestores públicos e para o público geral.

Nesta sprint, o foco foi **estruturar o repositório**, **definir a arquitetura de dados** (Bronze–Silver–Gold) e **preparar o esqueleto de back-end e front-end** que serão evoluídos nas próximas iterações.

2 Organização do repositório

Antes desta sprint, o repositório GitHub estava vazio. Foram criadas as seguintes estruturas principais:

- **README.md**: arquivo principal de documentação do projeto, descrevendo objetivos, arquitetura, estrutura de pastas e instruções para executar o back-end e o front-end.
- **back-end/**: pasta dedicada à API e futuros serviços de back-end (FastAPI).
- **front-end/**: pasta dedicada ao dashboard web e protótipos de interface.
- **Sprints/**: pasta para documentação em LaTeX das sprints, facilitando o registro de decisões técnicas e evolução do projeto.

Essa organização reflete a separação de responsabilidades entre camada de dados, API e visualização, e prepara o terreno para uma evolução incremental do projeto.

3 Arquitetura de dados (Medallion)

Foi consolidada a adoção da arquitetura *Medallion*, com três camadas principais:

3.1 Camada Bronze

A camada Bronze é responsável pela ingestão bruta dos dados do SINAN via PySUS. Cada execução produz um lote (*batch*) armazenado em arquivos Parquet, organizados por doença, ano de origem e metadados de ingestão. A estrutura proposta segue o padrão:

```
bronze/sinan/  
  disease=<doenca>/  
    source_year=<ano>/  
      ingestion_date=<data>/  
        batch_id=<id>/data_<timestamp>.parquet
```

Também é previsto um arquivo de metadados (`metadata.json`) contendo informações como *batch id*, quantidade de linhas, colunas presentes e carimbo de data/hora da ingestão. Essa camada não realiza limpeza ou filtragem significativa; seu papel é preservar o dado o mais próximo possível da fonte.

3.2 Camada Silver

A camada Silver é responsável por aplicar regras de limpeza, seleção de colunas e normalização de códigos. As principais decisões desta sprint foram:

- Selecionar, para o caso de uso inicial, as colunas: `DT_NOTIFIC`, `SEM_NOT`, `NU_ANO`, `ID_MUNICIP`, `SG_UF_NOT`, `CLASSI_FIN` e `EVOLUCAO`.
- Converter `DT_NOTIFIC` para um tipo de data nativo, permitindo filtragem por ano, mês e janelas móveis (ex.: últimos 30 dias).
- Normalizar `ID_MUNICIP` para um código IBGE de 7 dígitos, compatível com as malhas territoriais oficiais do IBGE.
- Normalizar `SG_UF_NOT` para um código numérico de UF de 2 dígitos, mapeando para o nome/sigla correspondente.
- Criar colunas derivadas, como `MUNICIPIO` e `UF`, que facilitam a leitura por parte do usuário final.
- Definir chaves de deduplicação (por exemplo, `DT_NOTIFIC`, `SEM_NOT`, `ID_MUNICIP`, `SG_UF_NOT`) para remover notificações repetidas.

Os scripts existentes de Silver (fornecidos em arquivos separados) serão integrados em futuras sprints dentro de uma estrutura de pacote Python, aproveitando essa convenção de colunas e códigos.

3.3 Camada Gold

A camada Gold é responsável por produzir tabelas analíticas agregadas. Na Sprint 1 foi definido o formato-alvo para suportar o dashboard de mapa de calor:

- `disease_code` – código da doença (ex.: `deng`, `toxc`).
- `year` e `month` – ano e mês de referência.

- `cd_uf`, `nm_uf` – código IBGE e nome da unidade federativa.
- `cd_mun`, `nm_mun` – código IBGE e nome do município.
- `cases_total` – quantidade de notificações no período para aquela combinação (doença, UF/município, ano, mês).

Os arquivos Gold serão salvos em Parquet em uma estrutura como:

```
gold/sinan/disease=<doenca>/year=<ano>/gold_cases.parquet
```

Essa definição permite que a API responda rapidamente a consultas agregadas por UF e por município.

4 Back-end: esqueleto FastAPI

Como parte da Sprint 1, foi criado um esqueleto de aplicação FastAPI no diretório `back-end/`, com o seguinte conteúdo principal:

```
back-end/
  app/
    __init__.py
    main.py
  requirements.txt
```

O arquivo `requirements.txt` lista as dependências mínimas (`fastapi`, `uvicorn[standard]`, `pandas`, `pyarrow`).

No arquivo `app/main.py` foi implementado:

- Configuração básica da aplicação FastAPI, com título e descrição compatíveis com o projeto NSS.
- Função utilitária para listar doenças disponíveis com base na estrutura de diretórios em `gold/sinan/disease=*`.
- Função utilitária para carregar os arquivos Parquet da camada Gold para uma doença específica.
- Endpoint `/health`, que retorna o status de saúde da API.
- Endpoint `/diseases`, que lista as doenças presentes na camada Gold.
- Endpoint `/heatmap/uf`, que agrega a tabela Gold por UF (colunas `cd_uf` e `nm_uf`) e soma a coluna `cases_total`, com parâmetros opcionais de `year` e `month`.
- Endpoint `/heatmap/municipios`, que agrega por município (colunas `cd_mun` e `nm_mun`) e também aceita filtros de UF, ano e mês.

Esses endpoints ainda assumem que a pipeline Gold irá produzir colunas com nomes padronizados (`cd_uf`, `nm_uf`, `cd_mun`, `nm_mun`, `year`, `month`, `cases_total`), o que será implementado em sprints futuras.

5 Front-end: protótipo de dashboard

Na Sprint 1 foi criado um protótipo mínimo em HTML/JavaScript no diretório `front-end/`. O objetivo é validar a comunicação com a API antes de partir para um framework mais complexo.

O arquivo `index.html` contém:

- Um pequeno formulário para seleção de doença (por enquanto `deng` e `toxc`), ano e mês.
- Uma chamada `fetch` ao endpoint `/heatmap/uf` da API, exibindo o retorno em uma tabela (colunas: código da UF, nome da UF e número de casos).
- Estilização simples em CSS para tornar a visualização mais agradável.

O arquivo `front-end/README.md` documenta como executar o protótipo utilizando um servidor HTTP básico (por exemplo, `python -m http.server`).

Embora o mapa de calor propriamente dito (com malhas do IBGE e coloração por intensidade) ainda não tenha sido implementado, este protótipo já estabelece a integração back-end/front-end e serve como base para evoluções futuras.

6 Documentação de sprints

Por fim, foi criada a pasta `Sprints/`, com os arquivos:

- `sprint-1.tex`: este documento, descrevendo em detalhes as decisões e implementações da Sprint 1.
- `sprint-2.tex`: documento de planejamento da próxima sprint, contendo atividades previstas.

A ideia é compilar esses arquivos em PDF (por exemplo, com o `texpage` ou `pdflatex`) e armazenar os PDFs na mesma pasta, preservando o histórico de evolução do projeto.

7 Resumo da Sprint 1

De forma resumida, a Sprint 1 entregou:

- Organização inicial do repositório GitHub com separação clara entre back-end, front-end e documentação de sprints.
- Definição da arquitetura de dados baseada em camadas Bronze, Silver e Gold.
- Esqueleto funcional de API FastAPI com endpoints para saúde da API, listagem de doenças e agregações por UF e município.
- Protótipo estático de front-end consumindo o endpoint de UF.
- Estrutura de documentação em LaTeX para registrar decisões técnicas e planejamento.

Esses elementos criam uma base sólida sobre a qual será possível avançar para a implementação completa da pipeline de dados, do dashboard com mapa de calor e de funcionalidades adicionais em sprints futuras.